

Documento de implementação técnica

Grupo 10 : Gabriel Spisila, Gabriel Leite, Davi Miessa

1. Visão Geral da Arquitetura:

• Frontend:

- Single Page Application (SPA): Escolhemos fazer um SPA devido a velocidade de troca de paginas
- Decidimos não adicionar React, Vue ou outro framework assim para simplificar o desenvolvimento.
- Utilizamos o Vite para facilitar o desenvolvimento e por suas implementações com a biblioteca Lucide-Static
- O Lucide-Static foi usado para carregar ícones, junto com o Vite é possível apenas colocar a URL do ícone que deseja ser usado e ele será carregado na página.
- A biblioteca Zod foi escolhida para fazer validações dos dados entrados pelos usuários.
- A biblioteca dotenv guarda as informações da API do Supabase
- O Vitest é usado para fazer testes unitários no sistema

• Backend:

- Node.js com TypeScript
- Express e cors para lidar com o servidor que dá acesso a funções de admin do Supabase
- A própria biblioteca do Supabase é usada para fazer as requisições ao servidor

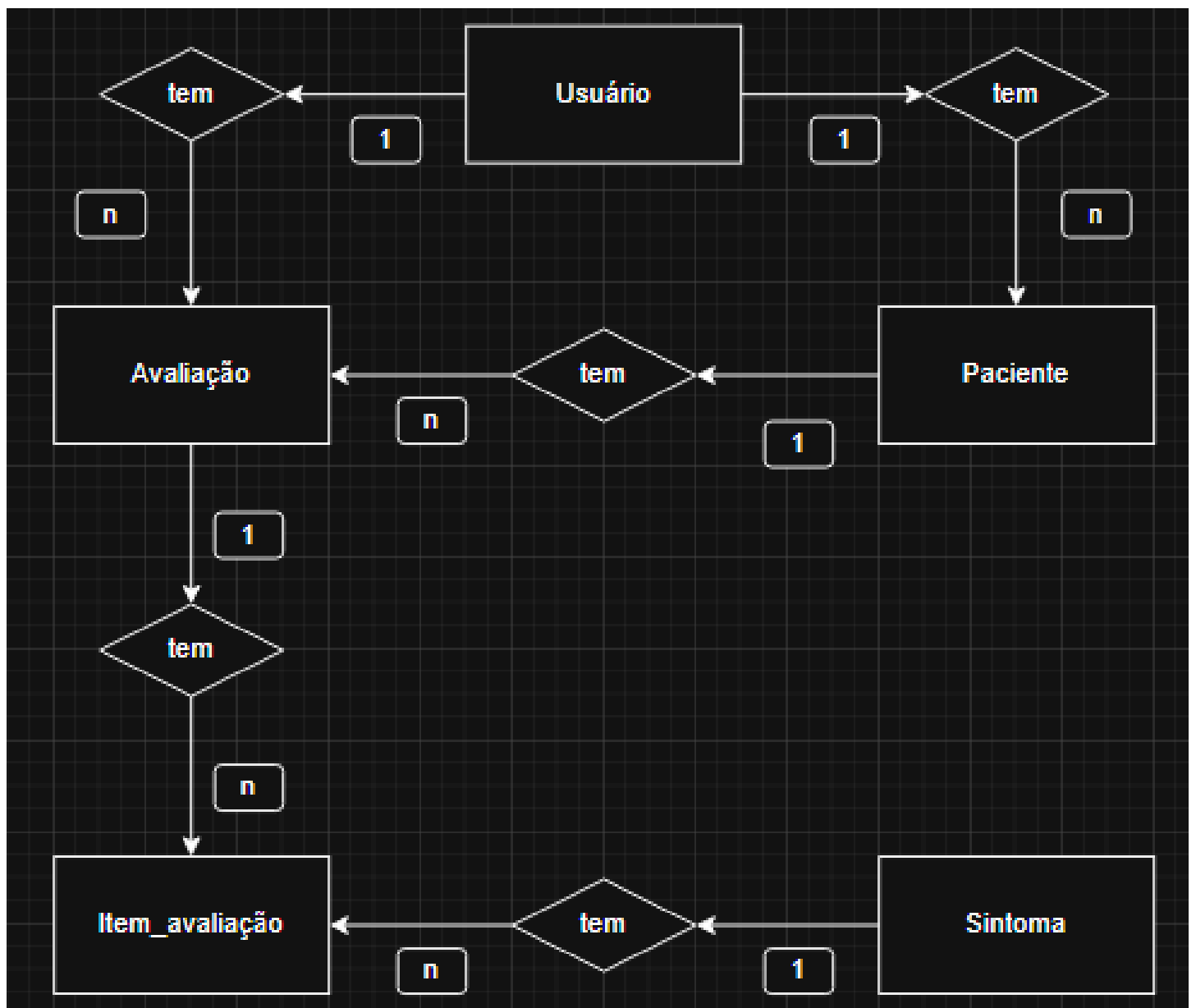
• Banco de dados:

- PostgreSQL hospedado no Supabase: O Supabase foi escolhido devido ao seu sistema de autorização robusto que já vem pronto e sua interface online que permite participação no desenvolvimento do banco de dados em equipe.

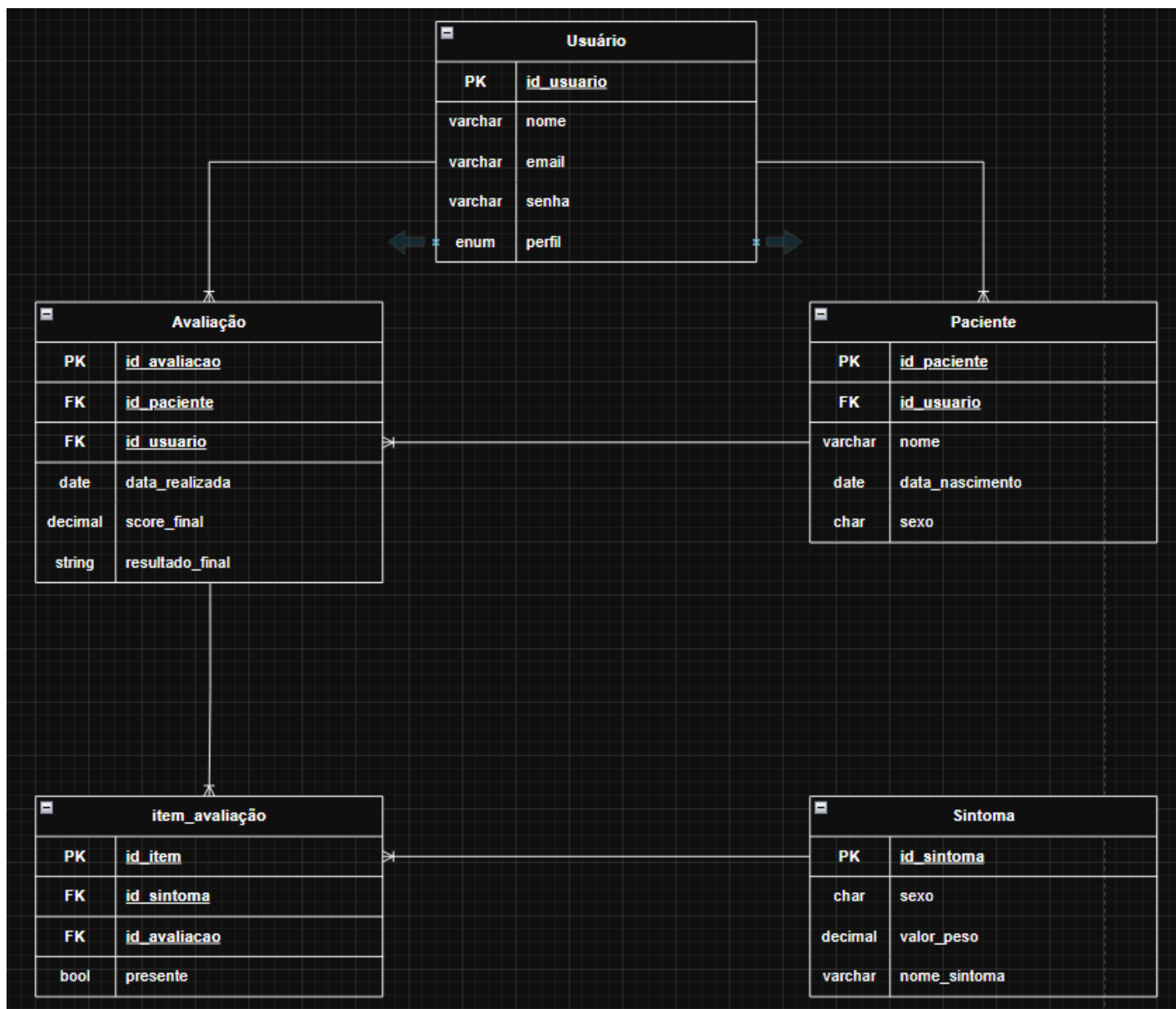
2. Modelagem do Banco de Dados:

• 1. MODELO CONCEITUAL E JUSTIFICATIVA DE ENTIDADES

- O presente documento descreve a arquitetura do banco de dados desenvolvido para o sistema de triagem clínica e avaliação do Transtorno do Espectro Autista (TEA). O sistema tem como objetivo fornecer aos profissionais da saúde ferramentas para registrar pacientes, gerenciar sintomas de base clínica e executar folhas de avaliação estruturadas gerando scores automatizados baseados em pesos científicos diferenciados por sexo.
- **1.1 Diagrama Conceitual e Modelo Lógico Físico (O Começo e o Fim)**
- Esta seção apresenta a evolução prática do banco de dados do sistema de triagem de TEA, partindo da ideia inicial até a estrutura final implementada na nuvem.
- **1.1.1 Modelo Conceitual (Visão Abstrata)**
- Mapeamento inicial das regras de negócio clínicas e cardinalidades básicas antes da definição de tabelas ou códigos.

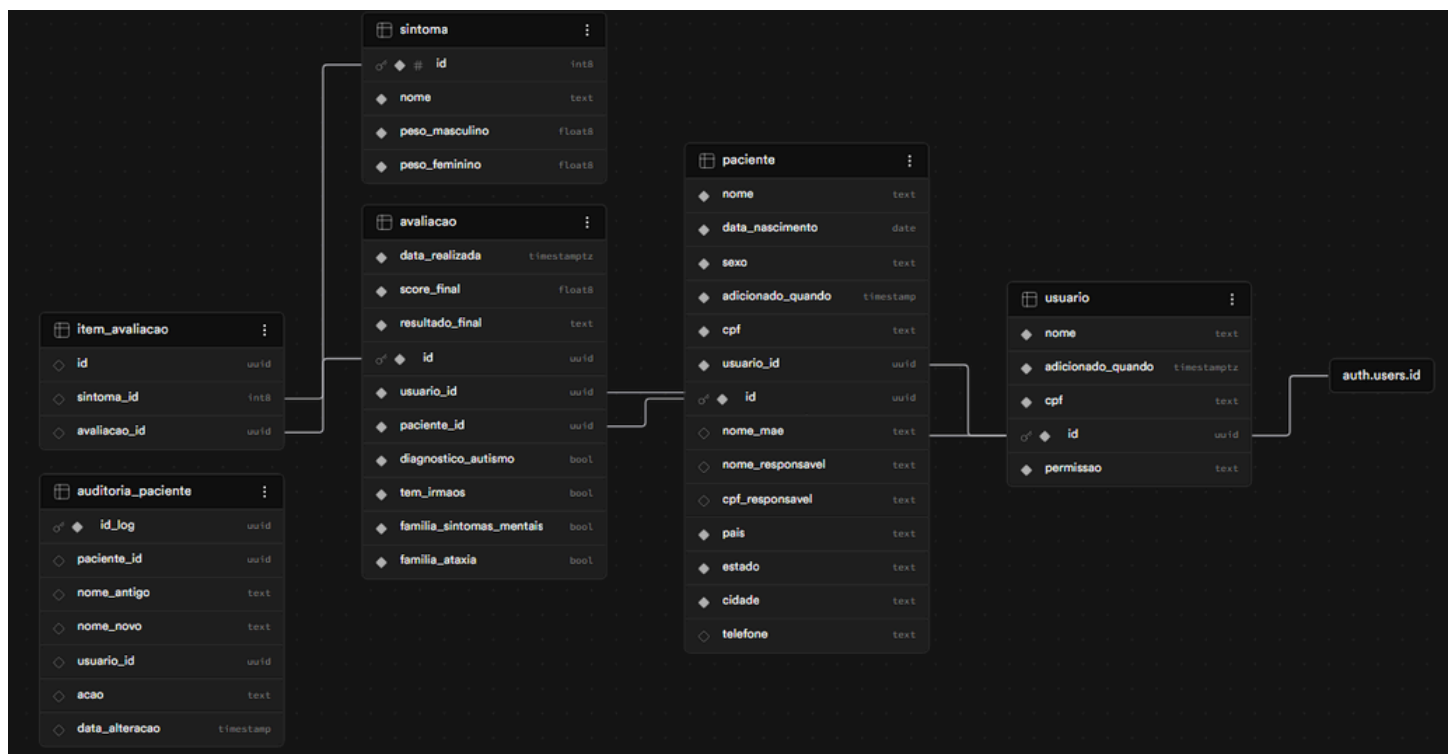


- **1.1.2 Modelo Lógico Inicial (O Começo)**
- Primeira modelagem relacional do sistema. Caracterizava-se pelo uso de chaves primárias numéricas sequenciais (id_usuario, id_paciente), tipagem genérica (varchar, decimal) e armazenamento local de senhas.



• 1.1.3 Modelo Lógico Físico do Supabase (O Fim)

- Arquitetura atual, definitiva e otimizada para o ecossistema **Supabase**. As principais melhorias aplicadas nesta versão final foram:
 - **Segurança:** Uso de uuid como chave primária para impedir a exposição de dados clínicos por URLs.
 - **Autenticação Nativa:** Vinculação direta com a tabela de login do Supabase (auth.users).
 - **Otimização:** Tipagem avançada do Postgres (text, float8, timestamptz).
 - **Escopo:** Inclusão de campos clínicos de anamnese e endereço, além de uma tabela automática para auditoria e logs de segurança.



• 1.2 Descrição das Entidades Principais

- **usuario:** Extensão do módulo de autenticação do Supabase. A sua chave primária (id) aponta diretamente para a tabela nativa auth.users(id). Armazena o nome, CPF e o nível de acesso através de uma string.
- **paciente:** Armazena o prontuário base estável do indivíduo (Nome, CPF, Data de Nascimento, Sexo Biológico) e dados dos responsáveis legais para casos de pacientes menores de idade.
- **sintoma:** Catálogo de referência dos sintomas clínicos. Para evitar a duplicação de registros, a tabela é unificada e possui duas colunas concorrentes de peso (peso_masculino e peso_feminino), visto que um mesmo sintoma tem impactos diferentes dependendo do sexo do paciente.
- **avaliacao:** Ficha que registra o evento de uma consulta num instante de tempo (timestampz). Guarda o score total calculado, o laudo gerado e o histórico de anamnese familiar (se tem irmãos, histórico de ataxia ou sintomas mentais na família).
- **item_avaliacao:** Tabela associativa pura criada exclusivamente para resolver o relacionamento Muitos-para-Muitos (N:N) entre Avaliações e Sintomas, registrando quais sintomas específicos foram detectados em cada consulta.
- **auditoria_paciente:** Tabela isolada para logs de segurança. É alimentada automaticamente por uma *Trigger* no banco de dados sempre que um prontuário de paciente é inserido, modificado ou eliminado.

• 1.3 Justificativa dos Relacionamentos e Cardinalidades

- **auth.users para usuario (1:1):** Cada conta criada no sistema de login do Supabase gera de forma unívoca um perfil nesta tabela. Evita a necessidade de gerir senhas ou criptografia na camada da aplicação.
- **usuario para paciente (1:N):** Um profissional de saúde pode cadastrar vários pacientes. Cada paciente possui a FK usuario_id obrigatória para identificar o seu criador, permitindo aplicar regras de isolamento de dados no sistema.
- **usuario para avaliacao (1:N):** Toda a avaliação clínica deve ser assinada por um profissional. Um médico realiza várias avaliações, mas uma avaliação

pertence a apenas um médico (usuario_id).

- **paciente para avaliacao (1:N) com ON DELETE CASCADE:** Um paciente pode ter várias avaliações ao longo do tempo. Foi configurada a restrição CASCADE: se um paciente for excluído, o seu histórico de consultas é eliminado automaticamente para evitar dados órfãos.
- **avaliacao e sintoma via item_avaliacao (N:N):** Uma consulta pode identificar vários sintomas, e um sintoma pode aparecer em várias consultas. A tabela item_avaliacao quebra esta relação complexa em duas chaves estrangeiras (FK) simples, garantindo a integridade referencial.

• **2. MODELO LÓGICO DE DADOS E NORMALIZAÇÃO (3FN)**

- O banco de dados foi normalizado até a Terceira Forma Normal (3FN), garantindo que não existam dependências parciais ou transitivas:
 - **Primeira Forma Normal (1FN):** Todos os atributos são estritamente atômicos (indivisíveis). Atributos compostos como o endereço do paciente foram desmembrados em colunas isoladas (cidade, estado). Não há grupos repetitivos.
 - **Segunda Forma Normal (2FN):** O banco cumpre a 1FN e todos os atributos não-chave dependem funcionalmente da totalidade da chave primária (PK).
 - **Terceira Forma Normal (3FN):** O banco cumpre a 2FN e nenhum atributo não-chave possui dependência transitiva em relação à chave primária por meio de outro atributo não-chave. Os pesos masculinos e femininos dependem unicamente e diretamente da chave de identificação do sintoma (id).

• **3. EVOLUÇÃO HISTÓRICA DO PROJETO**

- A evolução do banco de dados partiu de uma modelagem relacional genérica padrão para uma arquitetura totalmente otimizada dentro do ambiente **Supabase**. As principais alterações focaram na segurança dos dados e na eliminação de redundâncias.
- 3.1 Quadro de Mudanças de Tipos e Estrutura

Recurso / Campo	Como estava no Início	O que foi alterado para o Supabase	Justificativa Técnica
Identificadores (Chaves Primárias)	Uso de inteiros sequenciais simples (INT / BIGINT).	Alterado para uuid em todas as tabelas (utilizando gen_random_uuid()).	Evita que dados clínicos sejam expostos ou adivinhados sequencialmente através de URLs da aplicação.
Estrutura da Tabela de Sintomas	Modelagem clássica com sintomas duplicados em linhas separadas para o sexo Masculino e Feminino.	Unificação na tabela sintoma com duas colunas de pesos concorrentes (peso_masculino e peso_feminino).	Reduziu o tamanho da tabela pela metade. Agora, um único registro de sintoma atende a ambos os sexos, mudando apenas o valor do peso aplicado.
Coluna de Permissões (Usuário)	Campo de texto comum (TEXT ou VARCHAR).	Alterado para o tipo Enumerado Nativo (permissao_enum), aceitando apenas 'administrador' ou 'normal'.	Garante a integridade dos perfis direto no banco, impedindo erros de digitação na aplicação cliente.
Registro de Carimbo de Tempo	Campos de data comuns (DATE ou TIMESTAMP).	Alterado para timestamp tz (com fuso horário) nas tabelas	Garante precisão absoluta de quando a triagem foi feita, independente do fuso horário de onde o médico está logado.

• 3.2 Impacto Prático das Alterações no Trabalho

- A mudança mais crítica ocorreu na tabela associativa item_avaliacao. No modelo conceitual comum, ela usaria IDs numéricos comuns. Com a evolução para o Supabase, as chaves estrangeiras sintoma_id e avaliacao_id passaram a exigir validação estrita de tipos (int8 e uuid, respectivamente), impedindo que um checklist de sintomas seja salvo sem estar estritamente amarrado a uma consulta válida.

• 4. DICIONÁRIO DE DADOS ESTRUTURADO

• 4.1 Tabela: usuario

Nome da Coluna	Tipo de Dado	Chave	Restrições	Descrição / Finalidade
id	uuid	PK, FK	NOT NULL, REFERENCES auth.users(id)	ID do usuário, gerado e autenticado pelo Supabase.
nome	text	-	NOT NULL	Nome completo do profissional ou administrador .
cpf	text	-	NOT NULL, UNIQUE	CPF do usuário (chave única para evitar duplicados).
permissao	permissao_enum	-	NOT NULL	Nível de acesso: 'medico' ou 'administrador'.
adicionado_quando	timestampz	-	DEFAULT now()	Data e hora em que o usuário foi criado.

- **4.2 Tabela: paciente**

Nome da Coluna	Tipo de Dado	Chave	Restrições	Descrição / Finalidade
id	uuid	PK	NOT NULL, DEFAULT gen_random_uuid()	Identificador universal único do paciente.
usuario_id	uuid	FK	NOT NULL, REFERENCES usuario(id)	ID do profissional que cadastrou o paciente.
nome	text	-	NOT NULL	Nome completo do paciente.
data_nascimento	date	-	NOT NULL	Data de nascimento (para cálculo dinâmico da idade).
sexo	text	-	NOT NULL	Gênero biológico ('M' ou 'F') para cálculo do score.
cpf	text	-	NOT NULL, UNIQUE	CPF do paciente ou do seu responsável.
nome_mae	text	-	NOT NULL	Nome completo da mãe do paciente.
nome_responsavel	text	-	Opcional	Nome do responsável legal (se menor de idade).
cpf_responsavel	text	-	Opcional	CPF do responsável legal.

pais	text	-	DEFAULT 'Brasil'	País de residência.
estado	text	-	Opcional	Estado/UF de residência.
cidade	text	-	Opcional	Cidade de residência.
telefone	text	-	Opcional	Telefone principal de contato.
adicionado_qu ando	timestampz	-	DEFAULT now()	Data de criação do prontuário no banco.

• 4.3 Tabela: sintoma

Nome da Coluna	Tipo de Dado	Chave	Restrições	Descrição / Finalidade
id	int8	PK	NOT NULL, IDENTITY	ID incremental para indexação rápida dos sintomas.
nome	text	-	NOT NULL, UNIQUE	Descrição textual do sintoma clínico.
peso_masculino	float8	-	NOT NULL, DEFAULT 0.0	Peso do sintoma se o paciente for do sexo masculino.
peso_feminino	float8	-	NOT NULL, DEFAULT 0.0	Peso do sintoma se o paciente for do sexo feminino.

- **4.4 Tabela: avaliacao**

Nome da Coluna	Tipo de Dado	Chave	Restrições	Descrição / Finalidade
id	uuid	PK	NOT NULL, DEFAULT gen_random_uuid()	ID único da folha de consulta.
paciente_id	uuid	FK	NOT NULL, REFERENCES paciente(id) ON DELETE CASCADE	Paciente avaliado. Deleta o histórico se o paciente for excluído.
usuario_id	uuid	FK	NOT NULL, REFERENCES usuario(id)	Profissional de saúde que realizou o teste.
data_realizada	timestampz	-	DEFAULT now()	Carimbo de data e hora exata da consulta.
score_final	float8	-	NOT NULL, DEFAULT 0.0	Pontuação final somada com base nos sintomas.
resultado_final	text	-	NOT NULL	Laudo automático gerado (ex: "Encaminhar").
diagnostico_autismo	bool	-	DEFAULT false	Anamnese: histórico ou suspeita prévia de TEA.
tem_irmaos	bool	-	DEFAULT false	Anamnese: sinaliza se o paciente tem irmãos.
familia_sintomas_mentais	bool	-	DEFAULT false	Anamnese: histórico

				familiar de transtornos mentais.
familia_ataxia	bool	-	DEFAULT false	Anamnese: presença de casos de Ataxia na família.

• 4.5 Tabela: item_avaliacao

Nome da Coluna	Tipo de Dado	Chave	Restrições	Descrição / Finalidade
id	uuid	PK	NOT NULL, DEFAULT gen_random_uuid()	ID único da linha de associação.
avaliacao_id	uuid	FK	NOT NULL, REFERENCES avaliacao(id) ON DELETE CASCADE	Vinculo com a avaliação. Limpa o item se a avaliação for deletada.
sintoma_id	int8	FK	NOT NULL, REFERENCES sintoma(id)	ID do sintoma que foi identificado na consulta.

• 4.6 Tabela: auditoria_paciente

Nome da Coluna	Tipo de Dado	Chave	Restrições	Descrição / Finalidade
id_log	uuid	PK	NOT NULL, DEFAULT gen_random_uuid()	ID exclusivo do registro de log de auditoria.
paciente_id	uuid	-	NOT NULL	ID do paciente que sofreu a alteração.
nome_antigo	text	-	Opcional	Nome anterior (usado apenas em operações de UPDATE).
nome_novo	text	-	Opcional	Nome modificado/novo (usado em INSERT e UPDATE).
usuario_id	uuid	-	Opcional	ID do usuário que fez a alteração (se aplicável).
acao	text	-	NOT NULL	Operação realizada: 'INSERT', 'UPDATE' ou 'DELETE'.
data_alteracao	timestamp	-	DEFAULT now()	Data e hora exata em que o log foi gerado.

• 5. LÓGICA INTERNA: FUNCTIONS E TRIGGERS

- Para garantir a segurança, o desempenho e a automação dos processos clínicos diretamente no PostgreSQL do Supabase, foram desenvolvidos funções estruturadas e gatilhos nativos.
- 5.1 cadastrar_paciente_transacao

- Função estruturadas como transação atômica para garantir a consistência dos dados ao salvar uma nova consulta no sistema.
 - **Operação:** Insere o paciente, recupera o ID gerado, cria o registro de anamnese na tabela **avaliacao** e roda um laço para salvar os sintomas na tabela associativa **item_avaliacao**.

× Edit 'cadastrar_paciente_transacao' function

Name of function:
Name will also be used for the function name in postgres

Schema:
Tables made in the table editor will be in 'public'

Arguments
Arguments can be referenced in the function body using either names or numbers.

nome_paciente	text
data_nascimento_paciente	date
sexo_paciente	text
cpf_paciente	text
nome_mae_paciente	text
responsavel_paciente	text
cpf_responsavel_paciente	text
pais_paciente	text
estado_paciente	text
cidade_paciente	text
telefone_paciente	text
observacao_avaliacao	text
diagnostico_autismo_avaliacao	boolean
tem_irmaos_avaliacao	boolean
familia_sintomas_mentais_avaliacao	boolean
familiares_ataxia_avaliacao	boolean
avaliacao_sintomas	integer[]
usuario_id	uuid
score_fina_avaliacao	double

× Edit 'cadastrar_paciente_transacao' function

Definition
The language below should be written in plpgsql.

```

1
2 declare
3     paciente_id uuid;
4     avaliacao_id uuid;
5     sintoma_id_i integer;
6 begin
7     -- 1. Insert paciente pega o id
8     insert into paciente (usuario_id, nome, data_nascimento, sexo, cpf, nome_mae, nome_responsavel,
9     cpf_responsavel, pais, estado, cidade, telefone)
10    values (usuario_id, nome_paciente, data_nascimento_paciente, sexo_paciente, cpf_paciente,
11    nome_mae_paciente, responsavel_paciente, cpf_responsavel_paciente, pais_paciente, estado_paciente,
12    cidade_paciente, telefone_paciente)
13    returning id into paciente_id;
14
15 -- 2. Insert avaliacao pega o id
16 insert into avaliacao (usuario_id, paciente_id, score_final, resultado_final, diagnostico_autismo,
17 tem_irmaos, familia_sintomas_mentais, familia_ataxia)
18 values (usuario_id, paciente_id, score_fina_avaliacao, observacao_avaliacao,
19 diagnostico_autismo_avaliacao, tem_irmaos_avaliacao, familia_sintomas_mentais_avaliacao,
20 familiares_ataxia_avaliacao)
21 returning id into avaliacao_id;
22
23 -- 3. Insert items avaliacao
24 foreach sintoma_id_i in array avaliacao_sintomas loop
25     insert into item_avaliacao (sintoma_id, avaliacao_id)
26     values (sintoma_id_i, avaliacao_id);
27 end loop;
28
29 exception
30 when others then
31     raise exception 'Erro ao executar o cadastro completo: %', sqlerrm;
32 end;
33

```

- **5.2 gerar_auditoria_paciente e trigger_auditoria_paciente**
- Mecanismo automático de segurança para registrar o histórico de alterações em dados sensíveis de pacientes.
 - **Operação:** O gatilho monitora a tabela **paciente**. Após qualquer alteração, a função indentifica a ação realizada via **TG_OP** e salva o estado anterior e o novo na tabela **auditoria_paciente**.

× Edit 'gerar_auditoria_paciente' function

Name of function:
Name will also be used for the function name in postgres

Schema:
Tables made in the table editor will be in 'public'

Arguments
Arguments can be referenced in the function body using either names or numbers.

No argument for this function

× Edit 'gerar_auditoria_paciente' function

Definition
The language below should be written in plpgsql.

```

1
2 BEGIN
3
4     IF (TG_OP = 'INSERT') THEN
5         INSERT INTO public.auditoria_paciente (paciente_id, nome_antigo, nome_novo, usuario_id,acao)
6         VALUES (NEW.id, NULL, NEW.nome, NEW.usuario_id, 'CADASTRO');
7     END IF;
8
9     IF (TG_OP = 'UPDATE') THEN
10        INSERT INTO public.auditoria_paciente (paciente_id, nome_antigo, nome_novo, usuario_id,acao)
11        VALUES (NEW.id, OLD.nome, NEW.nome, NEW.usuario_id, 'ALTERAÇÃO');
12    END IF;
13
14    IF (TG_OP = 'DELETE') THEN
15        INSERT INTO public.auditoria_paciente (paciente_id, nome_antigo, nome_novo, usuario_id,acao)
16        VALUES (OLD.id, OLD.nome, NULL, OLD.usuario_id, 'EXCLUSÃO');
17    END IF;
18
19
20    RETURN NEW;
21 END;
22

```

Database Triggers
Execute actions automatically when database events occur

Data Event

schema public Q Search for a trigger Table Docs + New trigger

NAME	TABLE	FUNCTION	EVENTS	ORIENTATION	ENABLED
trigger_auditoria_paciente	paciente	gerar_auditoria_paciente	AFTER UPDATE AFTER DELETE AFTER INSERT	ROW	✓

- **5.3 get_dados_relatorio_especifico**
- Consulte em SQL puro criado para centralizar a busca de dados que alimentam os relatórios gerados pela aplicação.
 - **Operação:** Realiza junções relacionais para unificar os dados do paciente, do profissional e as respostas da triagem a partir do **avaliacao_id**.

× Edit 'get_dados_relatorio_especifico' function

Name of function
Name will also be used for the function name in postgres

Schema
Tables made in the table editor will be in 'public'

Arguments
Arguments can be referenced in the function body using either names or numbers.

× Edit 'get_dados_relatorio_especifico' function

Definition
The language below should be written in sql.

```

1
2
3  SELECT
4    p.nome AS paciente_nome,
5    p.data_nascimento AS paciente_data_nascimento,
6    p.sexo AS paciente_sexo,
7
8    u.nome AS usuario_nome,
9    u.id as usuario_id,
10
11    a.data_realizada AS data_avaliacao,
12    a.diagnostico_autismo AS tem_autismo,
13    a.temirmaos,
14    a.familia_sintomas_mentais,
15    a.score_final,
16    a.resultado_final
17
18  FROM avaliacao a
19  JOIN usuario u ON a.usuario_id = u.id
20  JOIN paciente p ON a.paciente_id = p.id
21
22  WHERE a.id = avaliacao_id
23
24

```

-
- **5.4 handle_new_user**
- Gatilho de automação integrado ao módulo de autenticação nativo do Supabase
 - **Operação:** É disparado no momento em que um novo login é criado, capturando os metadados encriptados **raw_user_meta_data** e inserindo o perfil de forma síncrona na tabela pública **usuario**.

×

Edit 'handle_new_user' function

Name of function

handle_new_user

Name will also be used for the function name in postgres

Schema

schema public

Tables made in the table editor will be in 'public'

Arguments

Arguments can be referenced in the function body using either names or numbers.

No argument for this function

Definition

The language below should be written in plpgsql.

```
1
2  begin
3      insert into public.usuario (id, nome, permissao, cpf)
4      values (
5          new.id,
6          new.raw_user_meta_data->>'nome',
7          new.raw_user_meta_data->>'perfil',
8          new.raw_user_meta_data->>'cpf'
9      );
10     return new;
11 end;
12
```

- **5.5 nova_avaliacao_transacao**
- Função procedural desenvolvida para salvar uma nova avaliação clínica e os seus sintomas associados de forma unificada.
 - **Operação:** A rotina recebe os dados da consulta e insere o registo principal na tabela **avaliacao**. Logo em seguida, utiliza um laço **FOREACH** para percorrer o array de sintomas recebido e popular de forma automática a tabela **item_avaliacao**. A função conta ainda com um bloco **EXCEPTION** que desfaz toda a operação em caso de erro, garantindo a segurança dos dados.

✕ Edit 'nova_avaliacao_transacao' function

Name of function:
Name will also be used for the function name in postgres

Schema:
Tables made in the table editor will be in 'public'

Arguments
Arguments can be referenced in the function body using either names or numbers.

p_observacao_avaliacao	text
p_diagnostico_autismo_avaliacao	boolean
p_tem_irmaos_avaliacao	boolean
p_familia_sintomas_mentais_avaliacao	boolean
p_familiares_ataxia_avaliacao	boolean
p_avaliacao_sintomas	integer[]
p_score_final_avaliacao	double
p_usuario_id	uuid
p_avaliacao_id	uuid
p_paciente_id	uuid

✕ Edit 'nova_avaliacao_transacao' function

Definition
The language below should be written in pLpgsqL.

```

1
2 declare
3   v_paciente_id uuid;
4   v_avaliacao_id uuid;
5   v_sintoma_id_i integer;
6   v_foto_avaliacao_i text;
7 begin
8
9   -- 2. Insert avaliacao pega o id
10  insert into avaliacao (
11    id,
12    usuario_id,
13    paciente_id,
14    score_final,
15    resultado_final,
16    diagnostico_autismo,
17    tem_irmaos,
18    familia_sintomas_mentais,
19    familia_ataxia)
20  values (
21    p_avaliacao_id,
22    p_usuario_id,
23    p_paciente_id,
24    p_score_final_avaliacao,
25    p_observacao_avaliacao,
26    p_diagnostico_autismo_avaliacao,
27    p_tem_irmaos_avaliacao,
28    p_familia_sintomas_mentais_avaliacao,
29    p_familiares_ataxia_avaliacao)
30  returning id into v_avaliacao_id;
31
32  -- 3. Insert itens avaliacao
33  foreach v_sintoma_id_i in array p_avaliacao_sintomas loop
34    insert into item_avaliacao (sintoma_id, avaliacao_id)
35    values (v_sintoma_id_i, v_avaliacao_id);
36  end loop;
37
38  -- 4. Insert fotos avaliacao
39  -- if fotos_paths is not null then
40  --   foreach v_foto_avaliacao_i in array fotos_avaliacao loop
41  --     insert into fotos_avaliacao (avaliacao_id, foto_path)
42  --     values (v_avaliacao_id, v_foto_avaliacao_i);
43  --   end loop;
44  -- end if;
45
46 exception
47 when others then
48   raise exception 'Erro ao executar o cadastro completo: %', sqlerrm;
49 end;
50

```

• 5.6 obter_pacientes_do_usuario

- Função procedural desenvolvida para listar os pacientes vinculados a um profissional específico.
 - **Operação:** A rotina recebe o ID do médico e monta uma busca dinâmica na tabela **paciente**. Através de blocos condicionais IF, o sistema aplica a ordenação escolhida e define a direção da busca. Por fim, realiza a paginação limitando o retorno a blocos de 24 registros.

✕ Edit 'obter_pacientes_do_usuario' function

Name of function:
Name will also be used for the function name in postgres

Schema:
Tables made in the table editor will be in 'public'

Arguments
Arguments can be referenced in the function body using either names or numbers.

medico_id_param	uuid
ascendente	boolean
ordenar_por	text
pagina	integer

✕ Edit 'obter_pacientes_do_usuario' function

Definition
The language below should be written in pLpgsqL.

```

1
2 DECLARE
3   query_texto TEXT;
4 BEGIN
5   query_texto := '
6     SELECT p.id, p.nome, p.data_nascimento, p.sexo, p.cpf
7     FROM paciente p
8     WHERE p.usuario_id = $1
9     ORDER BY ';
10  IF ordenar_por = 'nome' THEN
11    query_texto := query_texto || 'p.nome';
12  ELSIF ordenar_por = 'nascimento' THEN
13    query_texto := query_texto || 'p.data_nascimento';
14  ELSIF ordenar_por = 'sexo' THEN
15    query_texto := query_texto || 'p.sexo';
16  ELSE
17    query_texto := query_texto || 'p.nome';
18  END IF;
19  IF ascendente THEN
20    query_texto := query_texto || ' ASC';
21  ELSE
22    query_texto := query_texto || ' DESC';
23  END IF;
24
25  query_texto := query_texto || ' OFFSET ' || ( pagina - 1 ) * 24;
26
27  query_texto := query_texto || ' LIMIT 24; ';
28
29  RETURN QUERY EXECUTE query_texto USING medico_id_param;
30 END;
31

```

• 5.7 obter_relatorios_do_paciente

- Função procedural desenvolvida para listar o histórico de avaliações clínicas de um paciente específico.
 - **Operação:** A rotina recebe o ID do paciente e monta uma consulta dinâmica que realiza uma junção entre as tabelas **paciente** e **avaliacao**. Através de condicionais IF, o sistema filtra os registos pelo ID do **paciente**, aplica a ordenação configurada e define a direção ascendente ou descendente do resultado.

Edit 'obter_relatorios_do_paciente' function

Name of function:
Name will also be used for the function name in postgres

Schema:
Tables made in the table editor will be in 'public'

Arguments

Arguments can be referenced in the function body using either names or numbers.

paciente_id_param	uuid
ascendente	boolean
ordenar_por	text

Edit 'obter_relatorios_do_paciente' function

Definition

The language below should be written in plpgsql.

```

1
2 DECLARE
3     query_texto TEXT;
4 BEGIN
5     query_texto := '
6         SELECT p.nome, p.data_nascimento, p.sexo, a.data_realizada, a.score_final, a.id
7         FROM paciente p
8         JOIN avaliacao a ON a.paciente_id = p.id
9         WHERE p.id = $1
10        ORDER BY ';
11    IF ordenar_por = 'nome' THEN
12        query_texto := query_texto || 'p.nome';
13    ELSIF ordenar_por = 'nascimento' THEN
14        query_texto := query_texto || 'p.data_nascimento';
15    ELSIF ordenar_por = 'sexo' THEN
16        query_texto := query_texto || 'p.sexo';
17    ELSE
18        query_texto := query_texto || 'p.nome';
19    END IF;
20    IF ascendente THEN
21        query_texto := query_texto || ' ASC';
22    ELSE
23        query_texto := query_texto || ' DESC';
24    END IF;
25    RETURN QUERY EXECUTE query_texto USING paciente_id_param;
26 END;
27
28

```

• 5.8 obter_relatorios_do_paciente_administrador

- Função procedural desenvolvida para listar o histórico de avaliações de um paciente com permissões de administrador, exibindo também o nome do profissional que realizou o atendimento.
 - **Operação:** A rotina recebe o ID do paciente e constrói uma consulta dinâmica que realiza a junção entre as tabelas **paciente**, **avaliacao** e **usuario**. Através de condicionais IF, o sistema filtra pelo paciente correspondente, aplica a ordenação selecionada e define o sentido ascendente ou descendente do resultado.

✕ Edit 'obter_relatorios_do_paciente_administrador' function

Name of function:
Name will also be used for the function name in postgres

Schema:
Tables made in the table editor will be in 'public'

Arguments
Arguments can be referenced in the function body using either names or numbers.

paciente_id	uuid
ordenar_por	text
ascendente	boolean

✕ Edit 'obter_relatorios_do_paciente_administrador' function

Definition
The language below should be written in pLpgsqL.

```

1
2 DECLARE
3     query_texto TEXT;
4 BEGIN
5     query_texto := '
6         SELECT p.nome, u.nome, p.data_nascimento, p.sexo, a.data_realizada, a.score_final, a.id
7         FROM paciente p
8         JOIN avaliacao a ON a.paciente_id = p.id
9         JOIN usuario u ON a.usuario_id = u.id
10        WHERE p.id = $1
11        ORDER BY ';
12     IF ordenar_por = 'nome' THEN
13         query_texto := query_texto || 'p.nome';
14     ELSEIF ordenar_por = 'usuario' THEN
15         query_texto := query_texto || 'u.nome';
16     ELSEIF ordenar_por = 'data_nascimento' THEN
17         query_texto := query_texto || 'p.data_nascimento';
18     ELSEIF ordenar_por = 'sexo' THEN
19         query_texto := query_texto || 'p.sexo';
20     ELSEIF ordenar_por = 'data_avaliacao' THEN
21         query_texto := query_texto || 'a.data_realizada';
22     ELSEIF ordenar_por = 'score' THEN
23         query_texto := query_texto || 'a.score_final';
24     ELSE
25         query_texto := query_texto || 'p.nome';
26     END IF;
27     IF ascendente THEN
28         query_texto := query_texto || ' ASC';
29     ELSE
30         query_texto := query_texto || ' DESC';
31     END IF;
32
33     RETURN QUERY EXECUTE query_texto USING paciente_id;
34 END;
35

```

• 5.9 obter_todos_relatorios

- Função procedural desenvolvida para listar o histórico global de avaliações clínicas de todos os pacientes cadastrados no sistema.
 - **Operação:** A rotina constrói uma consulta dinâmica que realiza a junção entre as tabelas **paciente** e **avaliacao**. Através de condicionais IF, o sistema aplica a ordenação escolhida e define a direção ascendente ou descendente do resultado final.

✕ Edit 'obter_todos_relatorios' function

Name of function:
Name will also be used for the function name in postgres

Schema:
Tables made in the table editor will be in 'public'

Arguments
Arguments can be referenced in the function body using either names or numbers.

ascendente	boolean
ordenar_por	text
pagina	integer

✕ Edit 'obter_todos_relatorios' function

Definition
The language below should be written in pLpgsqL.

```

1
2 DECLARE
3     query_texto TEXT;
4 BEGIN
5     query_texto := '
6         select p.nome as nome_p, u.nome as nome_u, p.data_nascimento, p.sexo, a.data_realizada, a.
7         score_final, a.id
8         from paciente p
9         join avaliacao a on a.paciente_id = p.id
10        join usuario u on a.usuario_id = u.id
11        ORDER BY ';
12     IF ordenar_por = 'nome' THEN
13         query_texto := query_texto || 'p.nome';
14     ELSEIF ordenar_por = 'nascimento' THEN
15         query_texto := query_texto || 'p.data_nascimento';
16     ELSEIF ordenar_por = 'sexo' THEN
17         query_texto := query_texto || 'p.sexo';
18     ELSEIF ordenar_por = 'medico' THEN
19         query_texto := query_texto || 'u.nome';
20     ELSEIF ordenar_por = 'score' THEN
21         query_texto := query_texto || 'a.score_final';
22     ELSE
23         query_texto := query_texto || 'p.nome';
24     END IF;
25     IF ascendente THEN
26         query_texto := query_texto || ' ASC';
27     ELSE
28         query_texto := query_texto || ' DESC';
29     END IF;
30
31     query_texto := query_texto || ' OFFSET ' || ( pagina - 1 ) * 24;
32     query_texto := query_texto || ' LIMIT 24; ';
33
34     RETURN QUERY EXECUTE query_texto;
35 END;
36

```

• 5.10 pegar_dados_relatorios

- Função procedural desenvolvida para listar o histórico de avaliações clínicas vinculadas a um profissional específico cadastrado no sistema.

- **Operação:** A rotina recebe o ID do profissional e monta uma consulta dinâmica que realiza a junção entre as tabelas **paciente** e **avaliacao**. Através de condicionais IF, o sistema filtra os registros pelo ID do usuário logado, aplica a ordenação configurada e define a direção ascendente ou descendente do resultado.

Edit 'pegar_dados_relatorios' function

Name of function:
Name will also be used for the function name in postgres

Schema:
Tables made in the table editor will be in 'public'

Arguments
Arguments can be referenced in the function body using either names or numbers.

usuario_id	uuid
ordenar_por	text
ascendente	boolean

Edit 'pegar_dados_relatorios' function

Definition
The language below should be written in plpgsql.

```

1
2 DECLARE
3     query_texto TEXT;
4 BEGIN
5     query_texto := '
6         SELECT p.nome, p.data_nascimento, p.sexo, a.data_realizada, a.score_final, a.id
7         FROM paciente p
8         JOIN avaliacao a ON a.paciente_id = p.id
9         WHERE a.usuario_id = $1
10        ORDER BY ';
11    IF ordenar_por = 'nome' THEN
12        query_texto := query_texto || 'p.nome';
13    ELSIF ordenar_por = 'data_nascimento' THEN
14        query_texto := query_texto || 'p.data_nascimento';
15    ELSIF ordenar_por = 'sexo' THEN
16        query_texto := query_texto || 'sexo';
17    ELSIF ordenar_por = 'data_avaliacao' THEN
18        query_texto := query_texto || 'a.data_realizada';
19    ELSIF ordenar_por = 'score' THEN
20        query_texto := query_texto || 'a.score_final';
21    ELSE
22        query_texto := query_texto || 'p.nome';
23    END IF;
24    IF ascendente THEN
25        query_texto := query_texto || ' ASC';
26    ELSE
27        query_texto := query_texto || ' DESC';
28    END IF;
29    RETURN QUERY EXECUTE query_texto USING usuario_id;
30 END;
31
32

```

• 5.11 pegar_dados_todos_relatorios

- Função procedural desenvolvida para listar o histórico global de avaliações clínicas de todos os pacientes para a visão de um administrador do sistema.
 - **Operação:** A rotina monta uma consulta SQL dinâmica que realiza a junção entre as tabelas paciente e avaliacao. Através de estruturas condicionais IF, o sistema aplica os critérios de ordenação enviados por parâmetro e define a direção ascendente ou descendente antes de executar e retornar a listagem completa dos dados.

✕ Edit 'pegar_dados_todos_relatorios' function

Name of function
Name will also be used for the function name in postgres

Schema 
Tables made in the table editor will be in 'public'

Arguments
Arguments can be referenced in the function body using either names or numbers.

ordenar_por	text
ascendente	boolean

✕ Edit 'pegar_dados_todos_relatorios' function

Definition
The language below should be written in psql.

```
1
2 DECLARE
3     query_texto TEXT;
4 BEGIN
5     query_texto := '
6         SELECT p.nome, u.nome, p.data_nascimento, p.sexo, a.data_realizada, a.score_final, a.id
7         FROM paciente p
8         JOIN avaliacao a ON a.paciente_id = p.id
9         JOIN usuario u ON a.usuario_id = u.id
10        ORDER BY ';
11     IF ordenar_por = 'nome' THEN
12         query_texto := query_texto || 'p.nome';
13     ELSEIF ordenar_por = 'usuario' THEN
14         query_texto := query_texto || 'u.nome';
15     ELSEIF ordenar_por = 'data_nascimento' THEN
16         query_texto := query_texto || 'p.data_nascimento';
17     ELSEIF ordenar_por = 'sexo' THEN
18         query_texto := query_texto || 'sexo';
19     ELSEIF ordenar_por = 'data_avaliacao' THEN
20         query_texto := query_texto || 'a.data_realizada';
21     ELSEIF ordenar_por = 'score' THEN
22         query_texto := query_texto || 'a.score_final';
23     ELSE
24         query_texto := query_texto || 'p.nome';
25     END IF;
26     IF ascendente THEN
27         query_texto := query_texto || ' ASC';
28     ELSE
29         query_texto := query_texto || ' DESC';
30     END IF;
31
32     RETURN QUERY EXECUTE query_texto;
33 END;
34
```

• TABELA DE COMPONENTES E TECNOLOGIAS DO BANCO DE DADOS

Componente / Termo	O que significa na prática?	Onde foi aplicado no projeto?
Supabase	É a plataforma de Backend-as-a-Service (BaaS) escolhida para hospedar, gerenciar e prover a API do banco de dados na nuvem.	É o ecossistema geral onde todo o banco de dados do sistema de triagem de TEA está rodando.
PostgreSQL (Postgres)	É o motor de banco de dados relacional que roda por baixo dos panos do Supabase, cuidando do armazenamento físico e das queries.	É a linguagem e o padrão de banco utilizado (text, float8, timestampz, etc.).
PK (Primary Key)	Chave Primária. É o identificador único e obrigatório de cada linha de uma tabela. Nenhuma linha pode ter a PK repetida ou nula.	O campo id nas tabelas usuario, paciente, sintoma, avaliacao, e o campo id_item na tabela item_avaliacao.
FK (Foreign Key)	Chave Estrangeira. É uma coluna que cria um vínculo de integridade, apontando para a Chave Primária de outra tabela.	usuario_id (em paciente e avaliacao), paciente_id (em avaliacao), além de sintoma_id e avaliacao_id (em item_avaliacao).
UUID	<i>Universally Unique Identifier</i> . É um tipo de dado que gera um código alfanumérico aleatório de 36 caracteres impossível de adivinhar.	Utilizado como o tipo padrão de Chave Primária (PK) em quase todas as tabelas para garantir a segurança dos dados.
TEXT / STRING	Tipo de dado usado para armazenar cadeias de caracteres (textos) de tamanho variável sem limite fixo.	Nas colunas nome e cpf (em usuario e paciente), sexo (em paciente), nome (em sintoma) e resultado_final (em avaliacao).
FLOAT8	Tipo de dado numérico de ponto flutuante com precisão dupla (8 bytes). Usado para números decimais precisos.	Nas colunas de peso (peso_masculino e peso_feminino em sintoma) e na nota da consulta (score_final em avaliacao).
TIMESTAMPZ	Tipo de dado de data e hora que inclui o fuso horário (<i>Time Zone</i>). Grava o instante exato do servidor.	Nas colunas adicionado_quando (em usuario e paciente) e data_realizada (em avaliacao).
DATE	Tipo de dado usado para armazenar apenas o ano, mês	Na coluna data_nascimento da tabela paciente.

	e dia, sem registrar o horário.	
ENUM (Enumeração)	Tipo de dado customizado que restringe o valor da coluna a uma lista fixa de opções pré-definidas.	Na coluna permissao da tabela usuario, limitando o sistema a aceitar estritamente 'administrador' ou 'normal'.
ON DELETE CASCADE	Restrição de integridade que apaga automaticamente os registros filhos se o registro pai for deletado.	Aplicado na coluna avaliacao_id da tabela item_avaliacao para limpar o checklist caso a consulta seja excluída.
Tabela Associativa	Uma tabela intermediária que serve para quebrar uma relação de Muitos-para-Muitos (N:N) em duas relações simples de 1:N.	A tabela item_avaliacao , que conecta a lista de sintomas com as consultas médicas realizadas.

3. Engenharia de Roteamento e Fluxo do Client

- Nosso site tem apenas um único arquivo html. Esse arquivo tem uma barra superior e uma div vazia com o id "app".
- Nosso site tem um arquivo "main.ts" que é responsável por inicializar o site e realizar o roteamento entre páginas
- Os pesos dos sintomas são guardados no backend e pegos para serem usados no site durante a inicialização
- Optamos por não usar React, Angular, Vue ou outra biblioteca assim para fazer o roteamento. Ao invés disso utilizamos um método de roteamento usando o histórico de navegação e a URL


```

type FuncaoInit = (container: HTMLElement) => void;

interface Pagina {
  path: string,
  init: FuncaoInit,
  perfil_necessario: string | null,
  titulo: string
}

const Paginas: Pagina[] = [
  { path: '/login_total',      init: init_login_total_page,      perfil_necessario: null,      titulo: "LOGIN" },
  { path: '/cadastro_paciente', init: init_cadastro_paciente_page, perfil_necessario: "medico",  titulo: "CADASTRO DE PACIENTE" },
  { path: '/cadastro_medico',   init: init_cadastro_medico_page,  perfil_necessario: "administrador", titulo: "CADASTRO DE USUARIO" },
  { path: '/cadastro_admin',   init: init_cadastro_admin_page,   perfil_necessario: "administrador", titulo: "CADASTRO DE ADMIN" },
  { path: '/opcoes_usuario',    init: init_opcoes_usuario_page,   perfil_necessario: "medico",  titulo: "OPÇÕES DO USUARIO" },
  { path: '/opcoes_admin',     init: init_opcoes_admin_page,     perfil_necessario: "administrador", titulo: "OPÇÕES DO ADMINISTRADOR" },
  { path: '/lista_pacientes',   init: init_lista_pacientes_page,  perfil_necessario: "medico",  titulo: "LISTA DE PACIENTES" },
  { path: '/relatorios_usuario', init: init_relatorios_usuario_page, perfil_necessario: "medico",  titulo: "RELATORIOS DO USUARIO" },
  { path: '/relatorios_admin',  init: init_relatorios_admin_page,  perfil_necessario: "administrador", titulo: "RELATORIOS DO ADMINISTRADOR" },
  { path: '/relatorio',        init: init_relatorio_page,        perfil_necessario: "autorizado", titulo: "RELATORIO" },
  { path: '/nova_avaliacao',    init: init_nova_avaliacao_page,    perfil_necessario: "medico",  titulo: "NOVA AVALIAÇÃO" }
]

let paths: string[] = Paginas.map(p => p.path);

export function navigateTo(url: string) {
  window.history.pushState(null, '', url);
  handleRouting();
}

```

- O type FuncaoInit guarda referencias as funções de inicialização de cada pagina

A interface Pagina guarda as informações sobre todas as paginas da aplicação.



- path: é o caminho da pagina, a parte que é escrita na URL
- init: é a função que é chamada que carrega a nova pagina
- perfil_necessario: é o nível de autorização que é necessário para poder acessar aquela pagina
- titulo: Na barra superior do site mostra o titulo da pagina, essa variável define isso
- A função "handle_routing" é responsável por realizar o roteamento.
 - Essa função pega o path da janela, a URL.
 - Ela usa a função supabase.auth.getSession(); para checar se o usuário esta logado no sistema
 - A função então limpa o html da div "app".
 - A URL é checada com o path de cada pagina registrada em Paginas.

- Se a pagina existir a permissão do usuario é checada contra a permissão necessaria da pagina, se elas forem iguais a função de inicialização é chamada e os conteudos da pagina são colocados dentro da div “app”. Se não o usuário tentou acessar uma pagina que nao existe e sera redirecionado a pagina de login.

4. Estratégia de Segurança e LGPD:

- A segurança dos dados do sistema é mantida através do RLS (Row Level Security) do Supabase, esse sistema determina quem tem acesso a quais tabelas e quando:
 - No caso do nosso sistema operações de INSERT e SELECT só podem ser feitas por usuarios autenticados.
 - A única tabela onde SELECT pode ser feito sem autenticação é a tabela “sintoma”, ja que os pesos dos sintomas são pegos do banco de dados assim que a pagina é inicializada
- O cadastro e login no sistema é feito através do sistema de auth do Supabase
- A função “supabase.auth.signInWithPassoword()” é usada para fazer o login no sistema no front end
- Para que um administrador possa fazer o cadastro de novos usuarios e administradores no sistema a função supabaseAdmin.auth.admin.createUser() foi usada
 - Essa função não pode ser usada no frontend por razões de segurança, então foi criado um servidor backend simples usando as bibliotecas cors e express.
 - O frontend manda um POST para a porta 3000 e o servidor recebe, e la dentro ele tem as permissões para executar essa função e criar um novo usuário no sistema